

A Local Hybrid Retrieval-Augmented Assistant for Customer Support Ticket Triage

Juan Pablo Arnedo

Master of Artificial Intelligence, Atlantis University

MAI 600: Natural Language Processing

Professor Ivan Suarez

August 26, 2026

Abstract

Support teams route every incoming ticket to a queue by hand. This project builds a triage assistant that runs entirely on a laptop: a retrieval step supplies the organisation's routing conventions, and a locally served three-billion-parameter model reads the ticket and writes a structured triage with citations. The system was evaluated on 50 held-out tickets from a public synthetic dataset of customer support tickets, against a no-retrieval baseline and against a majority-class floor, using McNemar's exact test for paired comparisons. Retrieval raised routing accuracy from 0.00 to 0.30 against a 0.10 floor, correcting 15 tickets and breaking none ($p = 0.0001$). The mechanism is not better reasoning: without retrieval the model does not produce queue names that exist, because the set of queues an organisation operates is local knowledge that no amount of model scale supplies. A second finding concerns the evaluation rather than the system. The dataset is generated by paraphrasing templates, and the median cosine similarity between a test ticket and its nearest neighbour in the raw corpus is 0.906. Scaling the knowledge base over that corpus raises queue accuracy to 0.68, which is not retrieval succeeding but the system locating a reworded copy of the test ticket; excluding neighbours above 0.85 collapses the figure to 0.12. The pipeline therefore de-duplicates before indexing and asserts the separation. The final system is a drafting aid for human reviewers, not an autonomous router.

Keywords: retrieval-augmented generation, small language models, local inference, ticket triage, evaluation, data contamination

A Local Hybrid Retrieval-Augmented Assistant for Customer Support Ticket Triage

Introduction

Every ticket that reaches a support desk is read by a person who decides three things before any work begins: which team owns it, what kind of request it is, and how urgent it is. That triage is repetitive, it is inconsistent between agents and across shifts, and it is a bottleneck — a misrouted ticket can sit in the wrong queue for a day before anyone notices.

The obvious application of a language model here runs into an equally obvious obstacle. Ticket text contains customer names, telephone numbers, account identifiers and descriptions of internal systems. Sending that to a hosted API is a decision about data governance that many support organisations are not willing to make, and in regulated sectors cannot make. A system that is useful in practice has to run where the data already is.

This project builds that system and, more importantly, measures whether it works. The intended user is a support agent or team lead who wants a first-pass triage to review rather than a decision made for them. The value is not full automation, which the results below do not support, but a consistent starting point produced in under two seconds on hardware the team already owns.

Background

Retrieval-augmented generation addresses a specific limitation of language models: they cannot know facts that were not in their training data, and an organisation's internal conventions are exactly such facts (Lewis et al., 2020). Rather than encoding knowledge in weights, the system retrieves relevant passages at inference time and conditions generation on them. This

matters here because support queues are renamed and reorganised, and any approach that bakes them into parameters goes stale.

Retrieval quality depends on how text is represented. Sparse lexical methods match on shared vocabulary, which is a weak signal when a customer describes a problem in their own words and an agent answers in the language of the resolution. Dense sentence embeddings map semantically similar text to nearby vectors regardless of surface form (Reimers & Gurevych, 2019), and approximate nearest-neighbour indexes make search over such vectors practical (Johnson et al., 2019).

Parameter-efficient fine-tuning offers an alternative: low-rank adapters modify a small number of parameters to teach a model a repeated behaviour or output format (Hu et al., 2021). The two techniques answer different questions, and part of this project's contribution is deciding empirically which applies to which field of the triage output.

On evaluation, the discipline that matters most in a small classroom study is comparing against a floor rather than against another version of the same system, and reporting paired comparisons with a significance test rather than a difference in point estimates (Dietterich, 1998). Both practices were absent from the Module 7 prototype and both changed the conclusions.

Problem Statement

Given the subject and body of an incoming support ticket, produce a routing queue drawn from the organisation's real taxonomy, a ticket type, a priority, a draft first response, and citations to the past cases that justify the decision — with all processing local to the analyst's machine, and with an evaluation credible enough to say whether the system does anything a trivial rule could not.

Data Description

The data is the Customer IT Support ticket dataset published on Kaggle (tobiasbueck, 2025), chosen because it is public and synthetic and therefore carries no real customer information. It contains 20,000 tickets in English and German. Restricting to English rows with a complete subject, body, agent answer, queue, type and priority leaves 10,888 tickets across ten support queues.

One row is one support ticket. The subject and body are what the customer wrote; the answer field is the reply a support agent actually sent, which is what makes the dataset usable as a citable knowledge base rather than only as labelled training data. The queue field names one of ten routing destinations, type is one of Incident, Request, Problem or Change, and priority is low, medium or high.

The queue distribution is heavily skewed. Technical Support holds 3,119 tickets and General Inquiry holds 148, a ratio of roughly twenty to one. The knowledge base is therefore built with a balanced sample per queue, so that retrieval is not biased toward whichever queue merely contributed more material.

The most consequential property of this dataset is one that was not noticed until the final milestone. It was generated by paraphrasing templates, and it contains large families of near-identical tickets. Measured over the raw corpus, the median cosine similarity between a test ticket and its nearest neighbour is 0.906, and 26 of 50 test tickets have a neighbour above 0.90. These are not two customers reporting similar problems; they are the same ticket reworded. One pair, at a similarity of 0.992, differs only in the words “Query” versus “Enquiry” and the insertion of “on this”.

The consequence is quantified in Table 1. Predicting a ticket's queue by retrieving its single nearest neighbour from the raw corpus succeeds 68% of the time. That number does not

measure routing; it measures the system finding a paraphrase of the question and copying its answer. Excluding neighbours above successively lower similarity thresholds removes the effect and the apparent capability with it.

Table 1

Queue prediction accuracy by nearest-neighbour retrieval, as near-duplicates are excluded

De-duplication threshold	k = 1	k = 5	k = 15
None (raw corpus)	0.68	0.46	0.28
Exclude similarity ≥ 0.95	0.54	0.38	0.24
Exclude similarity ≥ 0.90	0.28	0.22	0.20
Exclude similarity ≥ 0.85	0.12	0.18	0.22

The corpus is therefore de-duplicated before anything else touches it, at a threshold of 0.90, retaining 7,163 of 10,888 tickets (65.8%). One representative is kept per near-duplicate family. The notebook then asserts that no test ticket has a knowledge-base neighbour at or above the threshold; the highest observed value is 0.899, and a violation fails the run rather than producing a footnote.

Methodology

The pipeline is: incoming ticket $\rightarrow$ de-duplicated corpus $\rightarrow$ queue documents $\rightarrow$ dense embedding and vector search $\rightarrow$ candidate ranking $\rightarrow$ prompt assembly with cited context $\rightarrow$ local generation $\rightarrow$ automatic evaluation against the dataset's labels.

Fifty tickets are held out, five from each queue, so that no queue is missing from the evaluation. The remaining 7,113 form the pool from which the knowledge base is drawn. The separation is strict: if a test ticket's own case is in the corpus, the system retrieves itself and every metric is meaningless.

The knowledge base is ten documents, one per queue, assembled from 1,782 past cases in total — up to 200 per queue, capped by availability for the smaller queues. Structuring the corpus by queue rather than as loose tickets mirrors how a help desk stores knowledge and makes citations legible: the model cites a named queue playbook rather than an opaque ticket identifier.

What gets embedded is the customer's description of the problem, not the agent's answer. An earlier version indexed resolution text and retrieved poorly, because agent replies are roughly half courtesy boilerplate and similarity was driven by the tone of a support email rather than by the problem. In one diagnosed case the top-ranked chunk scored 0.797 while containing no topical content at all, only an offer to schedule a call. The resolution is carried alongside each case and shown to the model as payload for drafting the reply, but it does not participate in search.

Each past case is a single chunk. A fixed-width word window straddles case boundaries and joins the tail of one problem to the head of an unrelated one, producing vectors that represent nothing in particular.

Chunks are embedded with all-MiniLM-L6-v2 into 384 dimensions and indexed in FAISS using inner product over L2-normalised vectors, which is cosine similarity. Each query retrieves five chunks. The retrieved chunks are collapsed into a ranked list of candidate queues by taking the mean similarity of the chunks each queue contributed. An offline ablation over the logged retrievals compared summing, taking the maximum, the mean, the count, the top chunk alone, and rank weighting; the mean ranked the correct queue first most often, at 0.24 against 0.18 for the sum that the prototype used. Summing rewards a queue for contributing more chunks irrespective of how well any of them matched.

Generation is served by Ollama, which exposes a local HTTP endpoint at port 11434, using Llama 3.2 at three billion parameters quantised to four bits (Meta AI, 2024). Prompts are sent by HTTP POST at temperature 0.2. The hardware is a MacBook Pro with an Apple M4 Pro and 48 GB of unified memory. A twelve-billion-parameter alternative was tested and averaged 110 seconds per ticket against under two, with no gain in routing accuracy, and was dropped.

System Architecture

Table 2

Pipeline stages from incoming ticket to evaluated output

Stage	What happens
Input	Ticket subject and body concatenated into a single query string
Corpus preparation	Full English corpus de-duplicated at cosine 0.90; one representative per family
Knowledge base	Ten queue documents, 1,782 past cases, balanced across queues
Indexing	One chunk per case; all-MiniLM-L6-v2 embeddings; FAISS inner-product index
Retrieval	Top-5 chunks; candidate queues ranked by mean similarity
Prompt	Retrieved cases with their queue, type and priority labels, plus a fixed output format
Generation	llama3.2:3b via Ollama at localhost:11434, temperature 0.2
Output	Queue, type, priority, citations, draft response
Evaluation	Automatic scoring against dataset labels; McNemar for paired comparisons

Implementation

Three system versions were run over the identical fifty tickets with the identical model, so that retrieval and prompt design are the only variables. V0 receives no retrieved context. V1 receives the retrieved cases and chooses a queue freely; this is the Module 7 prototype. V2 additionally receives the ranked candidate list and is instructed to select from it, defaulting to the top-ranked candidate and justifying any departure.

The list of valid queue names is deliberately withheld from V0. Knowing which queues an organisation operates is precisely the local knowledge retrieval is meant to supply, and handing it to the baseline for free would conceal the effect under measurement.

A majority-class reference is computed alongside the three: answer the most frequent label every time, with no model involved. It is the floor any real system must clear, and adding it was the single most useful change to this evaluation.

One implementation detail is worth recording for reproducibility. PyTorch and FAISS each ship their own OpenMP runtime, and on Apple Silicon calling into FAISS after PyTorch has run segfaults the process. Inside Jupyter this surfaces as a dead kernel with no traceback, which points the investigation toward memory exhaustion rather than a library conflict. Pinning FAISS to a single thread resolves it at negligible cost for an index of this size.

Results

Table 3
Evaluation metrics across system versions (n = 50 held-out tickets)

Metric	Majority class	V0 no retrieval	V1 RAG	V2 constrained
Routing accuracy	0.10	0.00	0.30	0.30
Queue validity	1.00	0.02	1.00	1.00
Type accuracy	0.40	0.62	0.70	0.72
Priority accuracy	0.40	0.42	0.46	0.46
Retrieval hit rate @5	—	—	0.50	0.50
Citation accuracy	—	—	0.40	0.34
Format adherence	—	1.00	1.00	1.00
Mean response time	—	0.97 s	1.62 s	1.80 s

Retrieval produces the central result. Routing accuracy rises from 0.00 without it to 0.30 with it, against a majority-class floor of 0.10. Paired over the same tickets, retrieval corrected fifteen that the baseline got wrong and broke none; McNemar's exact test gives $p = 0.0001$.

Queue validity explains the mechanism. Without retrieval the model almost never names a queue that exists — 0.02, a single ticket in fifty, and that one was still routed to the wrong queue. It does not appear to grasp that “queue” denotes a fixed routing destination, and instead returns a title summarising the ticket: “Excessive Billing Inquiry”, “Adobe Premiere Pro Crash Investigation”, “Digital Campaign Enhancement”. These read plausibly and a ticketing system would reject every one. With retrieval, validity is 1.00.

The constrained prompt introduced in this milestone is a null result. Against V1 it gains one ticket and loses one, $p = 1.000$. It was designed to close a Module 7 failure in which the model retrieved the correct evidence and then routed elsewhere anyway, defaulting to the most generic queue. That failure has indeed disappeared — generic-queue predictions fell from eight in fifty to two — but the knowledge base is what closed it, not the prompt. The constraint was compensating for a thin corpus, and with an adequate corpus it is redundant.

Table 4

Routing errors attributed to the pipeline stage that caused them

Stage	Errors	Share of errors
A — correct queue document never retrieved	25	71%
B — retrieved, but another queue ranked first	9	26%
C — ranked first, model chose otherwise	1	3%

Table 4 is the clearest statement of what changed between milestones. The prototype's bottleneck was stage C: the model held the right evidence and did not act on it, in four of ten

such cases. Here stage C accounts for one ticket in fifty. Seventy-one per cent of what remains is the retrieval step never surfacing the correct document, which is where further work belongs.

Per-queue accuracy is uneven in a way that is not random. Billing and Payments and Service Outages and Maintenance both reach 0.80; Customer Service and IT Support score 0.00. The queues that fail are the ones that overlap semantically — Customer Service, IT Support and Product Support resolve similar problems in similar language — and a human triage team would not agree unanimously on those boundaries either.

How Much of This Task Is Learnable

A routing accuracy of 0.30 means nothing on its own. Read against a majority-class floor of 0.10 it looks like the system does real work; read against a perfect router it looks like a failure. Neither reading is worth anything without knowing what the best available method reaches on the same tickets.

So one was trained. A logistic regression over sentence embeddings, fitted on all 7,113 de-duplicated training tickets together with their queue labels — far more supervision than the retrieval pipeline ever receives, since retrieval learns from no labels at all. On the same fifty held-out tickets it scores 0.34.

Table 5

Routing accuracy against the floor and against a supervised ceiling

Reference	Top-1	Top-3
Majority-class floor	0.10	—
V0, no retrieval	0.00	—
V2, this system	0.30	—
Supervised classifier, all-MiniLM-L6-v2	0.34	0.54
Supervised classifier, bge-base-en-v1.5	0.36	0.58
Supervised classifier, e5-base-v2	0.40	0.56

The pipeline reaches 88% of what a trained classifier extracts from the same text using the same encoder. The gap is about four tickets. Stronger encoders lift the ceiling a little and not far: doubling the embedding dimension with BGE-base buys two points, and E5-base gains six. E5 is the only lever tested in this project that produced a positive signal, and it was not pursued; it belongs in the next-steps list rather than in the results.

The queue label, then, is only weakly determined by the ticket text. The confusion matrix agrees — errors scatter across queues rather than clustering in one or two confusable pairs, which is what would be expected if the original labelling followed something the text does not carry. Four interventions were tested against this and none of them helped.

Table 6
Interventions tested against the ceiling, and their outcomes

Intervention	Outcome
Constrain the queue choice to retrieved candidates	$p = 1.000$, no effect
Retrieve 20 chunks instead of 5	hit rate 0.50 to 0.88, vote flat at 0.30
Merge overlapping queues into six groups	accuracy 0.38 but floor 0.30; margin falls from 3.0x to 1.3x
Suggest three candidate queues instead of one	covers 0.48 against a 0.30 floor, a worse ratio

Growing the knowledge base from 140 cases to 1,782 is the only change that moved the result. That is the honest summary of this system's performance: 0.30 sits close to what the data supports, and beating it requires better labels or signal the ticket text does not contain, not a better prompt.

Evaluation

Scoring is automatic and runs against the dataset's own labels, so figures reproduce on a re-run and there is no opportunity to grade generously. Eight metrics are reported. Retrieval hit rate asks whether the correct queue document appeared among the five retrieved. Routing

accuracy compares the predicted queue with the true one. Queue validity checks whether the predicted queue exists in the taxonomy at all, separating a wrong answer from an ungrounded one. Citation accuracy checks that the supporting document identifier appears in the model's citation field. Type and priority accuracy are exact matches. Format adherence checks that the requested fields were emitted. Response time is wall-clock seconds.

Paired comparisons use McNemar's exact test on the discordant pairs, which is the appropriate test when two systems are evaluated on the same items. Reporting a difference in point estimates without it was how the prototype reached two conclusions that a larger sample later reversed.

Improvement from the Prototype

Table 7

Changes from the Module 7 prototype and the evidence for each

Area	Prototype	Final version	Evidence
Data integrity	no duplicate check	de-duplicated at 0.90, asserted in code	highest test-to-KB similarity 0.899; raw corpus scored 0.68 from leakage alone
Knowledge base	140 cases, random	1,782 cases, balanced	generic-queue fallback fell from 8/50 to 2/50
Routing accuracy	0.25	0.30	$p = 0.0001$ against the no-retrieval baseline
Type accuracy	0.45, below its floor	0.72, floor 0.40	prototype finding was a small-sample artefact
Candidate ranking	sum of similarities	mean of similarities	offline ablation: 0.24 against 0.18
Test set	20 tickets	50 tickets	5 per queue rather than 2
Statistics	none	McNemar throughout	differences reported with p-values
Error analysis	counts only	attributed to pipeline stage	results/error_taxonomy.csv

One prototype conclusion is retracted here. Module 7 reported type accuracy of 0.45 against a 0.50 majority-class floor and concluded that type classification did not work at all. On

fifty tickets it scores 0.72 against a floor of 0.40. The earlier finding was an artefact of a twenty-ticket sample, and the correction is recorded rather than quietly dropped.

Discussion

What worked is narrower than it first appears, and more interesting. Retrieval does not make the model reason better about support tickets. It supplies a fact the model cannot otherwise have — the list of queues this organisation operates — and the effect is large and statistically clear. That is a precise argument for retrieval over fine-tuning in this setting: the knowledge is a set of names and conventions that change when the support organisation reorganises, and weights encoding them would go stale.

The result that did not improve is the more useful one to have measured. The constrained prompt, which was the planned contribution of this milestone, does nothing once the knowledge base is adequate. Had the knowledge base not been improved in the same cycle, the constraint would have appeared to work, and the wrong lesson would have been drawn about which intervention mattered.

The genuine surprise was the duplicate contamination. The planned improvement for this milestone was to scale the knowledge base from 140 cases to the full corpus, which would have raised routing accuracy from 0.25 to 0.68 and produced by far the most impressive number in the project. It would also have been meaningless. The system would not have learned to route; it would have found the test ticket's twin. This was caught only because a result that looked too good was questioned before it was written down.

The trade-off accepted throughout is capability for locality. A hosted frontier model would very likely route better. It would also mean sending customer telephone numbers and

account identifiers to a third party, which is the practice this project exists to avoid. Within that constraint, three seconds of laptop compute for a reviewable draft is a reasonable exchange.

Would the system be useful in practice? As a drafting aid, plausibly: it produces a valid, correctly formatted triage every time, in under two seconds, with citations an agent can check. As an autonomous router, no. Thirty per cent accuracy on a ten-way decision means seven tickets in ten arrive somewhere a human must correct.

Limitations and Risks

The test set is fifty tickets. That is sufficient to detect the retrieval effect, which is large, and insufficient for per-queue rates, which move by twenty points on a single error. Confidence intervals on those figures would be wide enough to be uninformative and are not reported.

Routing accuracy of 0.30 is three times the floor and nowhere near unsupervised operation. Priority does not work at all: 0.46 against a 0.40 floor, a difference of three tickets. Urgency depends on contract tier, service-level agreements and blast radius, none of which appear in ticket text, so neither the model nor a human reading only the ticket could recover it.

The labels themselves carry noise. Tickets asking how to configure an integration are filed as Change rather than Request, which places a ceiling on type accuracy that no model can exceed. Exact-match scoring against a single gold label therefore understates the system and overstates how well defined the task is.

Citation accuracy is a proxy: it verifies that the expected document identifier appears in the citation field, not that the cited passage supports the sentence attached to it. Genuine groundedness assessment requires manual review that was not performed.

The data is synthetic. Its near-duplicate structure is an artefact of template generation and would not appear in real support traffic in the same form, but neither would its regularity — real tickets are messier, and performance on them is unknown.

The principal risk in deployment is automation bias. A system that produces a confident, well-formatted answer every time invites acceptance without inspection, and this one is wrong on the queue seven times in ten.

Responsible AI and Ethics

All processing is local. No ticket text reaches a third party, which is the property that makes the system deployable in an environment handling customer data at all. The dataset used for development is public and synthetic, so no real customer information was involved in building or evaluating it.

Outputs are drafts for human review. The system should not close, route or reply to a ticket unattended, and any deployment should present its output as a suggestion with the retrieved evidence visible so that the reviewer can check the basis for the recommendation rather than only the recommendation.

Bias enters through the knowledge base. The corpus is drawn from past resolutions, so historical routing habits — including bad ones — are what the system reproduces. The queue distribution is skewed twenty to one, and an unbalanced knowledge base would amplify that; the balanced sampling used here is a partial mitigation, not a solution.

On transparency, every figure in this article is generated from the evaluation CSVs by script, and the de-duplication guarantee is an assertion that fails the run rather than a claim in prose. The intent is that a reader can re-run the notebook and obtain the same numbers, including the unflattering ones.

Conclusion

This project built a hybrid triage assistant that runs entirely on a laptop, combining dense retrieval over a knowledge base of past resolutions with a locally served three-billion-parameter model. Retrieval raised routing accuracy from 0.00 to 0.30 against a 0.10 floor, with $p = 0.0001$, and the mechanism is specific: it supplies the organisation's queue taxonomy, which the model has no other way to know.

Two things improved that were not the system. The evaluation acquired a trivial floor, paired significance testing and an error taxonomy attributing failures to pipeline stages; and the corpus acquired a de-duplication guard after the discovery that near-duplicate paraphrases were capable of inflating queue accuracy from 0.12 to 0.68. The second of these invalidated the improvement that had been planned for this milestone.

What was learned is that aggregate metrics concealed every substantive problem in this project. The boilerplate retrieval failure, the missing trivial baseline, a conclusion reversed by a larger sample, and the duplicate contamination were all found by reading individual cases after a summary table had looked acceptable.

The system reaches 88% of a supervised classifier trained on every label in the corpus, which is the sense in which 0.30 is close to finished rather than merely low. The next step is not a better prompt. Seventy-one per cent of remaining errors are the retrieval step failing to surface the right document, so improving the corpus and the ranking is where effort belongs. Beyond that, the evidence points toward adapter-based fine-tuning for the type field specifically, where the taxonomy is fixed and retrieval demonstrably adds little; training examples are prepared in the repository for that experiment, and no claim is made about its outcome.

References

- tobiasbueck. (2025). Customer IT support — ticket dataset (Version 14) [Data set]. Kaggle.
<https://www.kaggle.com/datasets/tobiasbueck/multilingual-customer-support-tickets>
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923.
<https://doi.org/10.1162/089976698300017197>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv*.
<https://arxiv.org/abs/2106.09685>
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
<https://doi.org/10.1109/TBDATA.2019.2921572>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Meta AI. (2024). Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>

Appendix A: Course Module Evidence

The project notebook closes with a section producing one artefact per course module, so that the path from each topic to the delivered system can be inspected rather than taken on trust. The artefacts are in the repository at the paths below.

Table A1
Course modules, their artefacts, and what each one settled

Module	Artefact	What it settled
1. Tokenisation and context limits	results/module1_token_budget.csv	Chunks average 75 tokens against the encoder's 256-token limit; 6 of 1,782 truncate. Truncation is ruled out as a source of retrieval error.
2. Sampling and temperature	results/module2_sampling_comparison.csv	Over six runs, temperatures 0.0 and 0.2 repeat themselves while 0.7 and 1.0 do not. An earlier three-run pass saw 0.2 vary, so the boundary sits near 0.2 and this sample size cannot place it precisely.
3. Attention as evidence selection	images/module3_evidence_map.png	Makes visible which queue the retrieved evidence favours and by what margin, which is where 71% of routing errors originate.
4. Prompting and structured output	prompts/baseline_prompt_template.txt, prompts/improved_rag_prompt_template.txt	Both templates exported so the difference between the baseline and the final prompt can be read without executing the notebook.
5. Local serving and benchmarking	results/module5_local_slm_benchmark.csv	0.9 seconds and 88 output tokens per second per triage; prompts average 1,074 tokens, comfortably inside the context window, so retrieval depth is not limited by it.
6. Retrieval-augmented generation	results/retrieved_chunks.csv	The retrieval pipeline described in the Methodology section.
7. Fine-tuning readiness	results/module7_rag_vs_finetuning_decision.csv, data/training_examples.jsonl	The RAG-or-fine-tuning decision is recorded per output field rather than for the system as a whole, because the measurements point different ways for different fields.
8. Integration and evaluation	results/summary_metrics.csv, images/evaluation_chart.png	The evaluation reported in the Results section.

The Module 7 table is the one that carries an argument. Queue prediction goes to retrieval because the knowledge involved is organisation-specific and changes when the support team reorganises. Type is the fine-tuning candidate: a fixed four-way taxonomy that retrieval

measurably fails to improve, since passing the retrieved cases' type labels into the prompt changed the score by nothing. Priority receives neither treatment, because the information that determines urgency is not present in the ticket text at all and no method operating on that text can recover it.